

BMI Calculator Programs: Overview and Comparison

Software Construction and Development - Lab Work 01

1. Overview of Both Programs

This document provides a technical breakdown and comparative analysis of two Python implementations of a BMI and Ideal Weight Calculator. Both programs process three user inputs (Age, Weight in kg, Height in cm) and compute four outputs (BMI score, WHO Weight Category, User Age, and Recommended Healthy Weight Range).

A. User-Created Version (`bmi_calculator_me.py`)

- Design Philosophy: Minimalist, clean, and direct procedural implementation.
- Structure: Uses a simple helper function `calculate_bmi()` that returns a tuple of metrics.
- Documentation: Zero code comments or docstrings, producing uncluttered code.
- Input Handling: Assumes standard valid user inputs directly converted using `float()` and `int()`.

GitHub Code Link (`bmi_calculator_me.py`):

GitHub Repository Work01 Directory:

<https://github.com/m-usman-k/Software-Construction-Development/tree/main/Work01>

Direct File Link:

https://github.com/m-usman-k/Software-Construction-Development/tree/main/Work01/bmi_calculator_me.py

B. AI-Generated Version (`bmi_calculator_ai.py`)

- Design Philosophy: Robust, production-grade script focusing on defensive programming.
- Structure: Uses typed functions returning structured dictionaries, encapsulated in a try-except block.
- Documentation: Full docstrings, type annotations, section dividers, and detailed code comments.
- Input Handling: Handles non-numeric inputs gracefully and checks for positive numbers.

GitHub Code Link (`bmi_calculator_ai.py`):

GitHub Repository Work01 Directory:

<https://github.com/m-usman-k/Software-Construction-Development/tree/main/Work01>

Direct File Link:

https://github.com/m-usman-k/Software-Construction-Development/tree/main/Work01/bmi_calculator_ai.py

2. Feature Comparison Table

Feature / Attribute	User Version (bmi_calculator_me.py)	AI Version (bmi_calculator_ai.py)
Code Length	Concise (~30 lines)	Detailed (~70 lines)
Documentation	None (Clean code, zero comments)	Full module docstrings and notes
Input Validation	Basic (Assumes valid input)	Robust (try-except and > 0 checks)
Type Annotations	No explicit type hints	Explicit Type Annotations (float, int)
Data Return Type	Tuple (bmi, category, min, max)	Dictionary with named keys
Console Output	Simple standard text output	Formatted ASCII decorative borders
Error Resiliency	Crashes on invalid user input	Catches errors gracefully

3. Pros and Cons Analysis

User Version (bmi_calculator_me.py)

Pros:

- Extremely simple to read, write, and understand for beginners.
- Zero visual noise or overhead, focusing purely on core mathematical logic.
- Fast execution and minimal file footprint.

Cons:

- Will throw runtime ValueError exceptions if the user inputs invalid text.
- Lack of comments or docstrings makes it less self-explaining in team settings.

AI Version (bmi_calculator_ai.py)

Pros:

- Production-ready error handling prevents application crashes.
- Self-documenting code with docstrings and type annotations for easy maintenance.
- Structured dictionary output improves clarity when consuming calculation results.

Cons:

- Slightly more verbose (~70 lines vs ~30 lines).
- Additional code structure adds slight overhead for simple tasks.